```
import pandas as pd
```

```
from google.colab import drive
drive.mount("/content/drive", force_remount=True)
```

```
Mounted at /content/drive
```

```
import os
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
os.chdir("/content/drive/MyDrive/IDS400")
```

```
os.listdir()
```

```
['Walmart_Sales.csv',
 'Income_Data.csv',
 'StudentsPerformance.csv',
 'customer-status.csv',
 'sales.csv',
 'UIC2016Basketball.csv',
 'retail.txt',
 'output.txt']
```

```
Walmart_Sales = pd.read_csv("/content/drive/MyDrive/IDS400/Walmart_Sales.csv")
```

```
print(Walmart_Sales.head())
```

```
   Store        Date  Weekly_Sales  Holiday_Flag  Temperature  Fuel_Price  \
0      1  05-02-2010    1643690.90             0        42.31       2.572
1      1  12-02-2010    1641957.44             1        38.51       2.548
2      1  19-02-2010    1611968.17             0        39.93       2.514
3      1  26-02-2010    1409727.59             0        46.63       2.561
4      1  05-03-2010    1554806.68             0        46.50       2.625

          CPI  Unemployment
0  211.096358         8.106
1  211.242170         8.106
2  211.289143         8.106
3  211.319643         8.106
4  211.350143         8.106
```

```
Walmart_Sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6435 entries, 0 to 6434
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Store         6435 non-null   int64
 1   Date          6435 non-null   object
 2   Weekly_Sales  6435 non-null   float64
 3   Holiday_Flag  6435 non-null   int64
 4   Temperature   6435 non-null   float64
 5   Fuel_Price    6435 non-null   float64
 6   CPI           6435 non-null   float64
 7   Unemployment  6435 non-null   float64
dtypes: float64(5), int64(2), object(1)
memory usage: 402.3+ KB
```

```
Walmart_Sales['Date'] = pd.to_datetime(Walmart_Sales['Date'], format='%d-%m-%Y')
```

```
print(Walmart_Sales['Date'].dtype)
```

```
datetime64[ns]
```

```
Walmart_Sales['Year'] = Walmart_Sales['Date'].dt.year # Extracted the Year from the date column but kept the original date
print(Walmart_Sales[['Date', 'Year']].head())
```

```
        Date  Year
0 2010-02-05  2010
1 2010-02-12  2010
2 2010-02-19  2010
3 2010-02-26  2010
4 2010-03-05  2010
```

```
Walmart_Sales['Month'] = Walmart_Sales['Date'].dt.month # Extracted the month from the date column but kept the original date
print(Walmart_Sales[['Date', 'Month']].head())
```

```
        Date  Month
0 2010-02-05      2
1 2010-02-12      2
2 2010-02-19      2
3 2010-02-26      2
4 2010-03-05      3
```

```
print(Walmart_Sales.head())
```

```
   Store       Date  Weekly_Sales  Holiday_Flag  Temperature  Fuel_Price  \
0      1 2010-02-05    1643690.90             0        42.31       2.572
1      1 2010-02-12    1641957.44             1        38.51       2.548
2      1 2010-02-19    1611968.17             0        39.93       2.514
3      1 2010-02-26    1409727.59             0        46.63       2.561
4      1 2010-03-05    1554806.68             0        46.50       2.625

          CPI  Unemployment  Year  Month
0  211.096358         8.106  2010      2
1  211.242170         8.106  2010      2
2  211.289143         8.106  2010      2
3  211.319643         8.106  2010      2
4  211.350143         8.106  2010      3
```

1. Which stores are most sensitive to temperature changes?

Do we think temperature changes Yearly sales?

```
Monthly_Data = Walmart_Sales.groupby(['Store', 'Month']).agg({'Weekly_Sales':'mean', 'Temperature':'mean'}).reset_index()
print(Monthly_Data.head())
```

```
   Store  Month  Weekly_Sales  Temperature
0      1      1  1.400468e+06    47.181250
1      1      2  1.625442e+06    47.786667
2      1      3  1.567744e+06    59.553077
3      1      4  1.544510e+06    67.392143
4      1      5  1.542111e+06    73.890000
```

```
Monthly_Data['All_Time_Sales_In_Millions'] = Monthly_Data['Weekly_Sales'] / 1_000_000 # Divided the millions for easier visualization and renamed the column
print(Monthly_Data[['Store', 'Month', 'All_Time_Sales_In_Millions']])
```

```
     Store  Month  All_Time_Sales_In_Millions
0        1      1                    1.400468
1        1      2                    1.625442
2        1      3                    1.567744
3        1      4                    1.544510
4        1      5                    1.542111
..     ...    ...                         ...
535     45      8                    0.736108
536     45      9                    0.723762
537     45     10                    0.737609
538     45     11                    0.877810
539     45     12                    1.067003

[540 rows x 3 columns]
```

```
Monthly_Temp = Walmart_Sales.groupby(['Store', 'Month']).agg({'Temperature': 'mean'}).reset_index() # Average Temp Per Store Per Month
print(Monthly_Temp.head())
```

```
   Store  Month  Temperature
0      1      1    47.181250
1      1      2    47.786667
2      1      3    59.553077
3      1      4    67.392143
4      1      5    73.890000
```

```
Monthly_Filtered_By_Store = pd.merge(Monthly_Data[['Store','Month','All_Time_Sales_In_Millions']],Monthly_Temp[['Store','Month','Temperature']],on=['Store','Month'],how='outer') # Merging bo
```

```
print(Monthly_Filtered_By_Store.head())
```

```
   Store  Month  All_Time_Sales_In_Millions  Temperature
0      1      1                    1.400468    47.181250
1      1      2                    1.625442    47.786667
2      1      3                    1.567744    59.553077
3      1      4                    1.544510    67.392143
4      1      5                    1.542111    73.890000
```

```
Sales_Correlation = Monthly_Filtered_By_Store.groupby('Store')['All_Time_Sales_In_Millions'].mean().reset_index() # Average revenue across the 3 years per store to prepare for the correlatio
Sales_Correlation.columns = ['Store', 'All_Time_Sales_In_Millions']
```

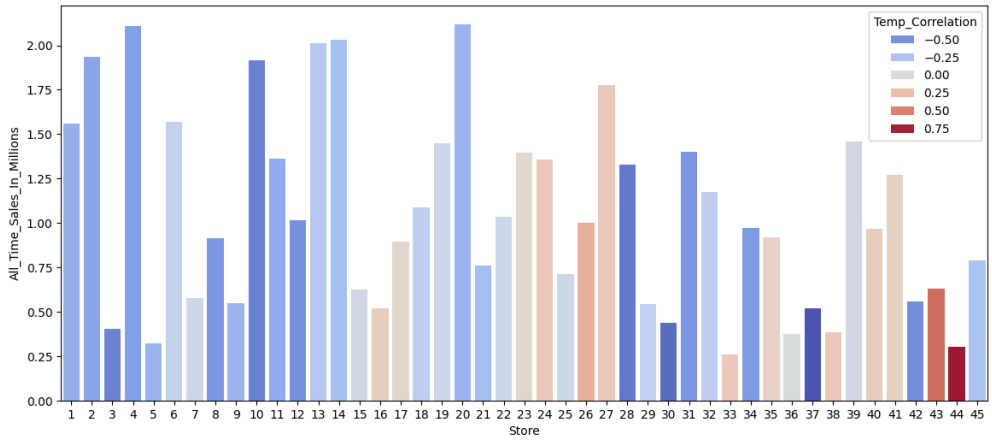
```
Store_Correlation = Monthly_Filtered_By_Store.groupby('Store').apply(lambda x: x['All_Time_Sales_In_Millions'].corr(x['Temperature'])).reset_index() # Correlation betweeen the overall store
Store_Correlation.columns = ['Store', 'Temp_Correlation']
```

Show hidden output

```
Filtered_Store_Data = pd.merge(Sales_Correlation, Store_Correlation, on='Store') # Ready To Visualize and filtered temp and sales data by store
```

```
plt.figure(figsize=(14,6))
sns.barplot(
    x='Store',
    y='All_Time_Sales_In_Millions',
    data=Filtered_Store_Data,
    palette=sns.color_palette("coolwarm", as_cmap=True),
    hue='Temp_Correlation',
    dodge=False)
```

```
<Axes: xlabel='Store', ylabel='All_Time_Sales_In_Millions'>
```



2. Which stores were the top 5 highest in sales from 2010 to 2012?

Do these Top 5 stores share common features such as maybe economic conditions, temperature, or unemployment rates?

```
Sales_Per_Store = Monthly_Filtered_By_Store.groupby('Store')['All_Time_Sales_In_Millions'].sum().reset_index()
print(Sales_Per_Store.head(5))
```

```
   Store  All_Time_Sales_In_Millions
0      1                  18.712779
1      2                  23.229828
2      3                   4.859519
3      4                  25.283036
4      5                   3.835325
```

```
Top_Five_Stores = Sales_Per_Store.sort_values(by='All_Time_Sales_In_Millions', ascending=False).head(5)
print(Top_Five_Stores)
```

```
    Store  All_Time_Sales_In_Millions
19     20                  25.435023
3       4                  25.283036
13     14                  24.357636
12     13                  24.141686
```

```
1       2                 23.229828
```

```
Mean_Of_Top_Five_Stores = Walmart_Sales.groupby('Store').agg({'Temperature': 'mean','CPI': 'mean','Unemployment': 'mean'}).reset_index()
print(Mean_Of_Top_Five_Stores.head())
```

```
   Store  Temperature         CPI  Unemployment
0      1    68.306783  215.996892      7.610420
1      2    68.216364  215.646311      7.623846
2      3    71.434196  219.391531      7.176986
3      4    62.253357  128.679669      5.964692
4      5    69.410140  216.565581      6.295406
```

```
Monthly_Top_Five = pd.merge(Top_Five_Stores,Mean_Of_Top_Five_Stores,on='Store',how='left')
print(Monthly_Top_Five.head())
```

```
   Store  All_Time_Sales_In_Millions  Temperature         CPI  Unemployment
0     20                   25.435023    55.451399  209.038131      7.371413
1      4                   25.283036    62.253357  128.679669      5.964692
2     14                   24.357636    57.790979  186.285678      8.648748
3     13                   24.141686    53.697133  128.679669      7.001825
4      2                   23.229828    68.216364  215.646311      7.623846
```

```
Avg_Temp = Walmart_Sales['Temperature'].mean()
Avg_Unemployment = Walmart_Sales['Unemployment'].mean()
Avg_CPI = Walmart_Sales['CPI'].mean()
print(Avg_Temp, Avg_Unemployment, Avg_CPI)
```

```
60.66378243978244 7.99915104895105 171.57839384878014
```

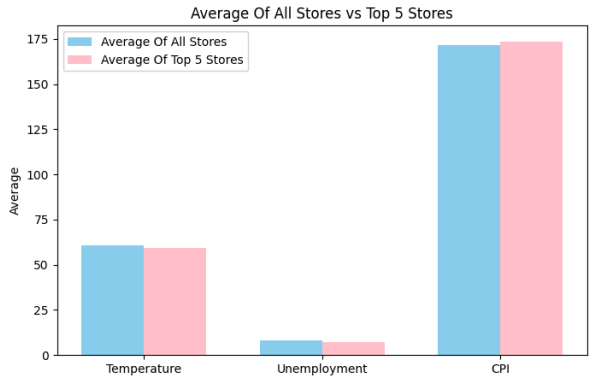
```
Avg_Temp_Monthly_Top_Five = Monthly_Top_Five['Temperature'].mean()
Avg_Unemployment_Monthly_Top_Five = Monthly_Top_Five['Unemployment'].mean()
Avg_CPI_Monthly_Top_Five = Monthly_Top_Five['CPI'].mean()
print(Avg_Temp_Monthly_Top_Five, Avg_Unemployment_Monthly_Top_Five, Avg_CPI_Monthly_Top_Five)
```

```
59.48184615384615 7.3221048951048955 173.66589164307692
```

```
Avg_For_All_3_Variables = ['Temperature', 'Unemployment', 'CPI']
All_Stores = [Avg_Temp, Avg_Unemployment, Avg_CPI]
Top5_Stores = [Avg_Temp_Monthly_Top_Five, Avg_Unemployment_Monthly_Top_Five, Avg_CPI_Monthly_Top_Five]

x = np.arange(len(Avg_For_All_3_Variables))
width = 0.35

plt.figure(figsize=(8,5))
plt.bar(x - width/2, All_Stores, width, label='Average Of All Stores', color='skyblue')
plt.bar(x + width/2, Top5_Stores, width, label='Average Of Top 5 Stores', color='pink')
plt.xticks(x, Avg_For_All_3_Variables)
plt.ylabel('Average')
plt.title('Average Of All Stores vs Top 5 Stores')
plt.legend()
plt.show()
```



3. Does unemployment influence the direction of Overall Revenue, if so how is this helpful?

```
Sales_Per_Store_All = Monthly_Data.sort_values(by='All_Time_Sales_In_Millions', ascending=False).reset_index(drop=True)
print(Sales_Per_Store_All.head(45))
```

```
    Store  Month  Weekly_Sales  Temperature  All_Time_Sales_In_Millions
0      20     12  2.676012e+06    33.949000                    2.676012
1       4     12  2.561152e+06    39.627000                    2.561152
2      14     12  2.555552e+06    37.899000                    2.555552
3      10     12  2.540212e+06    52.524000                    2.540212
4      13     12  2.538795e+06    30.238000                    2.538795
5       2     12  2.396695e+06    47.151000                    2.396695
6      20     11  2.364878e+06    45.566250                    2.364878
7       4     11  2.329136e+06    49.470000                    2.329136
8      14     11  2.246385e+06    47.447500                    2.246385
9      20      2  2.195947e+06    31.683333                    2.195947
10     13     11  2.189391e+06    39.947500                    2.189391
11      4      2  2.183575e+06    38.815833                    2.183575
12     10     11  2.177192e+06    61.242500                    2.177192
13     27     12  2.131044e+06    38.214000                    2.131044
14      2     11  2.108592e+06    58.070000                    2.108592
15     20      6  2.104559e+06    71.649231                    2.104559
16      4      8  2.085018e+06    80.146154                    2.085018
17     14      4  2.079593e+06    53.633571                    2.079593
18     10      2  2.078483e+06    54.146667                    2.078483
19     14      5  2.072984e+06    63.119167                    2.072984
20      4      6  2.063358e+06    79.838462                    2.063358
21     14      6  2.059911e+06    73.243077                    2.059911
22     20      4  2.056185e+06    52.176429                    2.056185
23      4     10  2.054847e+06    62.519231                    2.054847
24     20      5  2.053449e+06    62.896667                    2.053449
25     13      6  2.043500e+06    65.352308                    2.043500
26      4      3  2.041075e+06    53.245385                    2.041075
27     20      7  2.038563e+06    75.772143                    2.038563
28     20     10  2.037437e+06    55.546923                    2.037437
29      4      5  2.032642e+06    68.844167                    2.032642
30     20      8  2.032291e+06    73.057692                    2.032291
31      2      2  2.031104e+06    46.265000                    2.031104
32     14      2  2.029094e+06    34.698333                    2.029094
33     20      3  2.022135e+06    45.565385                    2.022135
34     13      8  2.018369e+06    78.243846                    2.018369
```

```
35      4      9  2.008490e+06    71.744615                    2.008490
36      4      4  2.006260e+06    61.820000                    2.006260
37     20      9  2.003479e+06    66.548462                    2.003479
38     14      3  1.998525e+06    46.560000                    1.998525
39      4      7  1.998500e+06    80.717857                    1.998500
40     13      7  1.971164e+06    77.962143                    1.971164
41     13      4  1.966663e+06    47.817857                    1.966663
42      6     12  1.966302e+06    50.155000                    1.966302
43     13      2  1.962969e+06    32.970000                    1.962969
44     13      5  1.962606e+06    54.226667                    1.962606
```

```
Unemployment_Per_Store_All = Walmart_Sales.groupby('Store')['Unemployment'].mean().reset_index()
Sales_Per_Store_All = Walmart_Sales.groupby('Store')['Weekly_Sales'].sum().reset_index()

print(Unemployment_Per_Store_All.head(5))
```

```
   Store  Unemployment
0      1      7.610420
1      2      7.623846
2      3      7.176986
3      4      5.964692
4      5      6.295406
```

```
Mean_All_Stores = Walmart_Sales.groupby('Store').agg({'Weekly_Sales': 'mean','Unemployment': 'mean'}).reset_index()

Mean_All_Stores['Top_5_Sales'] = Mean_All_Stores['Weekly_Sales'] / 1_000_000
Mean_All_Stores = Mean_All_Stores.drop(columns='Weekly_Sales')

print(Mean_All_Stores.head())
```

```
   Store  Unemployment  Top_5_Sales
0      1      7.610420     1.555264
1      2      7.623846     1.925751
2      3      7.176986     0.402704
3      4      5.964692     2.094713
4      5      6.295406     0.318012
```

```
Top_5_Unemployment_Stores = Mean_All_Stores.sort_values(by='Unemployment', ascending=False).head(5)
print(Top_5_Unemployment_Stores)
```

```
    Store  Unemployment  Top_5_Sales
11     12     13.116483     1.009002
37     38     13.116483     0.385732
27     28     13.116483     1.323522
42     43      9.934804     0.633325
33     34      9.934804     0.966782
```

```
Avg_Unemployment_Bottom_Five = Top_5_Unemployment_Stores.head(5)['Unemployment'].mean()
print(Avg_Unemployment_Bottom_Five)
```

```
11.84381118881119
```

```
Avg_Unemployment_Top_Five = Mean_All_Stores.tail(5)['Unemployment'].mean()
print(Avg_Unemployment_Top_Five)
```

```
8.131103496503496
```

```
Avg_Sales_Top_Five = Top_Five_Stores.head(5)['All_Time_Sales_In_Millions'].mean()
print(Avg_Sales_Top_Five)
```

```
24.48944198327967
```

```
Avg_Sales_Bottom_Five = Mean_All_Stores.tail(5)['Top_5_Sales'].mean()
print(Avg_Sales_Bottom_Five)
```

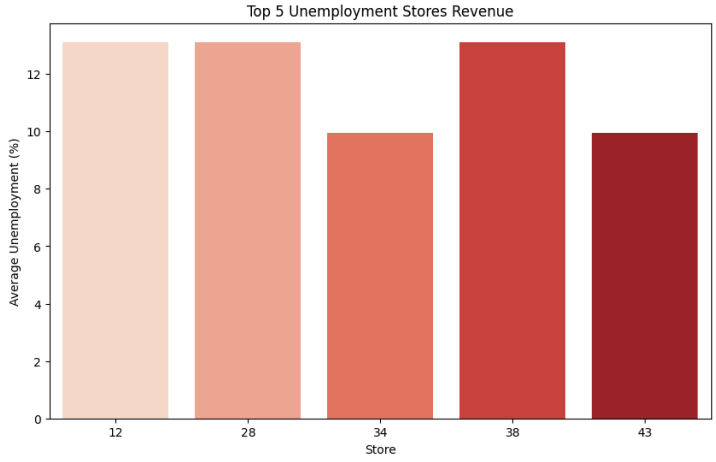
```
0.70931685593007
```

```
plt.figure(figsize=(10,6))
sns.barplot(
    x='Store',
    y='Unemployment',
    data=Top_5_Unemployment_Stores,
    palette='Reds')
plt.title('Top 5 Unemployment Stores Revenue')
plt.xlabel('Store')
plt.ylabel('Average Unemployment (%)')
plt.show()
```

```
/tmp/ipython-input-3039961924.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(
```




```
plt.figure(figsize=(10,6))
sns.barplot(
    x='Store',
    y='Top_5_Sales',
    data=Top_5_Unemployment_Stores,
    palette='Blues')
```

```
plt.title('Top 5 Unemployment Stores Revenue')
plt.xlabel('Store')
plt.ylabel('All Time Sales in Millions')
plt.show()
```

```
/tmp/ipython-input-2832462363.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(
```